U.E 5.1 - Intelligence Artificielle : Machine learning
Promotion FISE 2027

ENSTA BRETAGNE
2 Rue François Verny
29200 Brest Cedex 9, France

BARROS ALVES Luiz Felipe
luiz.barros@ensta.fr
BENIGNO DA SILVA Matheus Henrique
matheus.benigno@ensta.fr

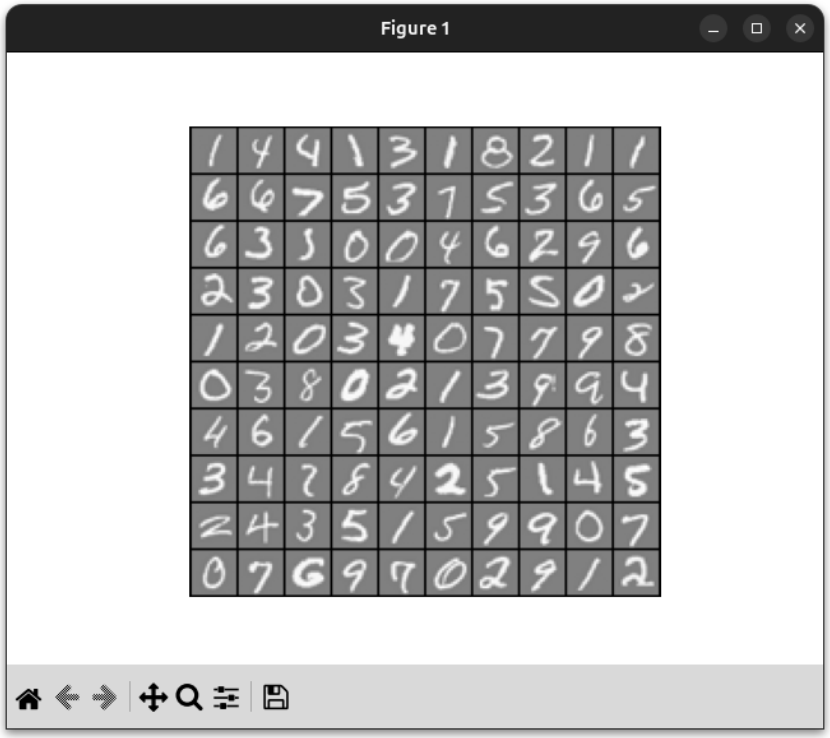
Sommaire

1 | 3 Visualizing the data **2**

2 | 4 Prediction by a neural network **2**
2.1 | 4.1 Model representation . . . 2
2.2 | 4.2 Feedforward Propagation and Prediction . . . 2

3 | 5 Learning a classifier with a neural Network **4**
3.1 | 5.1 Feedforward and cost function . . . 4
3.2 | 5.2 Regularized cost function . . . 5
3.3 | 5.3 Backpropagation . . . 6
3.3.1 | 5.3.1 Sigmoid gradient . . . 6
3.3.2 | 5.3.2 Random initialization . . . 6
3.3.3 | 5.3.3 The backpropagation algorithmn . . . 7
3.3.4 | 5.3.4 Gradient checking . . . 7
3.3.5 | 5.3.5 Regularized Neural Networks . . . 7
3.3.6 | 5.3.6 Sigmoid function . . . 8
3.4 | 5.4 Visualising the hidden layer . . . 8
3.5 | 5.5 Final analysis . . . 9
3.6 | 6 Some questions, you have to answer... . . . 16

1 | 3 Visualizing the data




```
 ------------------------

Loading and Visualizing Data ...

 ------------------------

Loading Saved Neural Network Parameters ...
```

2 | 4 Prediction by a neural network

2.1 | 4.1 Model representation

2.2 | 4.2 Feedforward Propagation and Prediction

```python
import numpy as np

from sigmoid import sigmoid

def predictNeuralNetwork(Theta1, Theta2, X):
    """ outputs the predicted label of X given the
    trained weights of a neural network (Theta1, Theta2)
    """

    # Useful values
    m, _ = X.shape
    num_labels, _ = Theta2.shape

    # ====================== YOUR CODE HERE ======================
    # Instructions: Complete the following code to make predictions using
```

```
    #               your learned neural network. You should set p to a
    #               vector containing labels between 1 to num_labels.
    #
    # Hint: The max function might come in useful. In particular, the np.argmax
    #       function can return the index of the max element, for more
    #       information see 'numpy.argmax' on the numpy website. If your examples
    #       are in rows, then, you can use np.argmax(probs, axis=1) to obtain the
    #       max for each row.
    #

    # =========================================================================

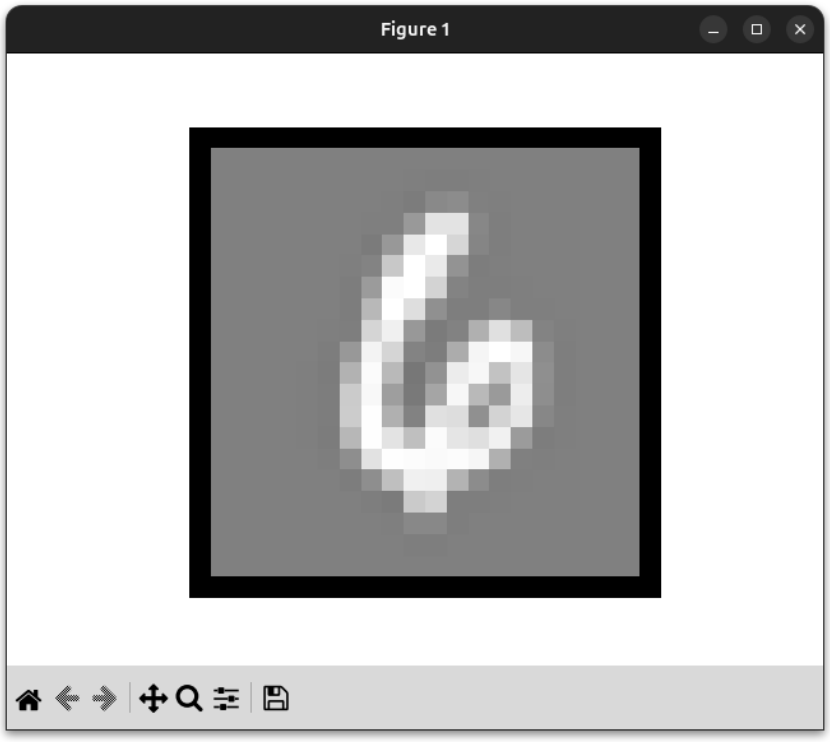
    z2 = X.dot(Theta1.T)
    a2 = sigmoid(z2)
    a2 = np.column_stack((np.ones((m, 1)), a2)) # bias unit

    z3 = a2.dot(Theta2.T)
    a3 = sigmoid(z3)

    p = np.argmax(a3, axis=1)

    # =========================================================================

    return p
```




```
 ------------------------

Loading Saved Neural Network Parameters ...

 ------------------------

Training Set Accuracy: %f 97.52
Expected training Set Accuracy: 97.5%
```

3 | 5 Learning a classifier with a neural Network

3.1 | 5.1 Feedforward and cost function

```python
import numpy as np

from sigmoid import sigmoid
from sigmoidGradient import sigmoidGradient


def nnCostFunction(nn_params, input_layer_size, hidden_layer_size, num_labels, X, y, Lambda):

    """computes the cost and gradient of the neural network. The
    parameters for the neural network are "unrolled" into the vector
    nn_params and need to be converted back into the weight matrices.

    The returned parameter grad should be a "unrolled" vector of the
    partial derivatives of the neural network.
        """

    # Reshape nn_params back into the parameters theta1 and theta2, the weight matrices
    # for our 2 layer neural network
    # Obtain theta1 and theta2 back from nn_params

    theta1 = nn_params[0:(hidden_layer_size*(input_layer_size+1))].reshape((input_layer_size+1),hidden_layer_size).T
    theta2 = nn_params[(hidden_layer_size*(input_layer_size+1)):].reshape((hidden_layer_size+1),num_labels).T

    # Setup some useful variables
    m, _ = X.shape

    # You need to return the following variables correctly
    J = 0
    theta1_grad = np.zeros(theta1.shape)
    theta2_grad = np.zeros(theta2.shape)

    # ====================== YOUR CODE HERE ======================
    # Instructions: You should complete the code by working through the
    #               following parts.
    #
    # Part 1: Feedforward the neural network and return the cost in the
    #         variable J. After implementing Part 1, you can verify that your
    #         cost function computation is correct by verifying the cost
    #         computed in ex3_nn_train.py
    #
    #   NOTE DE GLC: JE VOUS AI MIS LA CORRECTION DES PART 2 ET 3 dans la suite du code
    #
    #
    # Part 2: Implement the backpropagation algorithm to compute the gradients
    #         theta1_grad and theta2_grad. You should return the partial derivatives of
    #         the cost function with respect to theta1 and theta2 in theta1_grad and
    #         theta2_grad, respectively. After implementing Part 2, you can check
    #         that your implementation is correct by running checkNNGradients
    #
    #         Note: The vector y passed into the function is a vector of labels
    #               containing values from 1...K. You need to map this vector into a
    #               binary vector of 1's and 0's to be used with the neural network
    #               cost function.
    #
    #         Hint: We recommend implementing backpropagation using a for-loop
    #               over the training examples if you are implementing it for the
    #               first time.
```

```
    #
    # Part 3: Implement regularization with the cost function and gradients.
    #
    #         Hint: You can implement this around the code for
    #               backpropagation. That is, you can compute the gradients for
    #               the regularization separately and then add them to theta1_grad
    #               and theta2_grad from Part 2.
    #
    # =========================================================================


    # Add ones to the X data matrix
    a1 = np.column_stack((np.ones((m, 1)), X))

    z2 = a1.dot(theta1.T)
    a2 = sigmoid(z2)
    a2 = np.column_stack((np.ones((m, 1)), a2))

    z3 = a2.dot(theta2.T)
    a3 = sigmoid(z3)

    # Construct a 10xm "y" matrix with all zeros and only one "1" entry
    y_matrix = np.zeros((num_labels,m)) # A complter

    for i in range(m):
        y_matrix[y[i], i] = 1

    # Compute Cost
    J = (-1/m) * np.sum(y_matrix.T * np.log(a3) + (1 - y_matrix.T) * np.log(1 - a3)
        )
```

```
 ------------------------

Feedforward Using Neural Network ...

 ------------------------

Cost at parameters (loaded from ex3weights): 0.287629
(this value should be about 0.287629)
```

3.2 | 5.2 Regularized cost function

```
    # =========================================================================
    # PART 2:  Cost regularisation (skip it the first time)

    # Cost regularisation
    reg = (Lambda / (2 * m)) * (np.sum(theta1[:, 1:] ** 2)
                                + np.sum(theta2[:, 1:] ** 2))
    J = J + reg
```

```
 ------------------------

Checking Cost Function (w/ Regularization) ...

 ------------------------

Cost at parameters (loaded from ex3weights): 0.383770
(this value should be about 0.383770)

 ------------------------
```

3.3 | 5.3 Backpropagation

3.3.1 | 5.3.1 Sigmoid gradient

```python
from sigmoid import sigmoid

def sigmoidGradient(z):
    """computes the gradient of the sigmoid function
    evaluated at z. This should work regardless if z is a matrix or a
    vector. In particular, if z is a vector or matrix, you should return
    the gradient for each element."""

    # ====================== YOUR CODE HERE ======================
    # Instructions: Compute the gradient of the sigmoid function evaluated at
    #               each value of z (z can be a matrix, vector or scalar).

    s = sigmoid(z)
    g = s * (1 - s)

    # =============================================================

    return g
```

```
 ------------------------

Evaluating sigmoid gradient...
Sigmoid gradient evaluated at [1 -0.5 0 0.5 1]:[0.19661193 0.23500371 0.25
 0.23500371 0.19661193]
(this value should be about 0.196612 0.235004 0.250000 0.235004 0.196612)

 ------------------------
```

3.3.2 | 5.3.2 Random initialization

```python
import numpy as np

def randInitializeWeights(L_in, L_out):
    """randomly iniializes the weights of a layer with L_in incoming connections
        and L_out outgoing
    connections.
    Note that W should be set to a matrix of size(L_out, 1 + L_in) as the 1st
        column of W handles the "bias" terms"""

    # ====================== YOUR CODE HERE ======================================
    # Instructions: Initialize W randomly so that we break the symmetry while
    #               training the neural network.
    #
    # Note: The first row of W corresponds to the parameters for the bias units
    #

    epsilon_init = 0.12
    W = np.random.rand(L_out, 1 + L_in) * 2 * epsilon_init - epsilon_init

    # ============================================================================

    return W
```

3.3.3 | 5.3.3 The backpropagation algorithmn

3.3.4 | 5.3.4 Gradient checking

3.3.5 | 5.3.5 Regularized Neural Networks

```
 -------------------------

Initializing Neural Network Parameters ...

 -------------------------

Checking Backpropagation...
[[-9.27825235e-03 -9.27825236e-03]
 [ 8.89911959e-03  8.89911960e-03]
 [-8.36010761e-03 -8.36010762e-03]
```

```
 [ 1.49568335e-01  1.49568335e-01]
 [ 5.31542052e-02  5.31542052e-02]
 [ 4.65597186e-02  4.65597186e-02]]
The above two columns you get should be very similar.
(Left-Your Numerical Gradient, Right-Analytical Gradient)


If your backpropagation implementation is correct, then
 the relative difference will be small (less than 1e-9).

Relative Difference: 2.33368e-11


 -------------------------

Checking Backpropagation (w/ Regularization) ...
[[-9.27825235e-03 -9.27825236e-03]
 [ 8.89911959e-03  8.89911960e-03]
 [-8.36010761e-03 -8.36010762e-03]
```

```
 [ 1.32294136e-01  1.32294136e-01]
 [-4.52964427e-03 -4.52964427e-03]
 [ 1.50048382e-03  1.50048382e-03]]
The above two columns you get should be very similar.
(Left-Your Numerical Gradient, Right-Analytical Gradient)


If your backpropagation implementation is correct, then
 the relative difference will be small (less than 1e-9).

Relative Difference: 2.21026e-11

Cost at (fixed) debugging parameters (w/ lambda = 10): 0.576051 (this value shou
ld be about 0.576051)
```

```
-------------------------

Training Neural Network...
/home/matheus/Documentos/machine_learning/ex03_startercode/venv/lib/python3.12/s
ite-packages/scipy/optimize/_optimize.py:1704: OptimizeWarning: Maximum number o
f iterations has been exceeded.
  res = _minimize_cg(f, x0, args, fprime, callback=callback, c1=c1, c2=c2,
         Current function value: 0.426938
         Iterations: 50
         Function evaluations: 109
         Gradient evaluations: 109

-------------------------

Visualizing Neural Network...
Training Set Accuracy: 96.500000
```

3.3.6 | 5.3.6 Sigmoid function

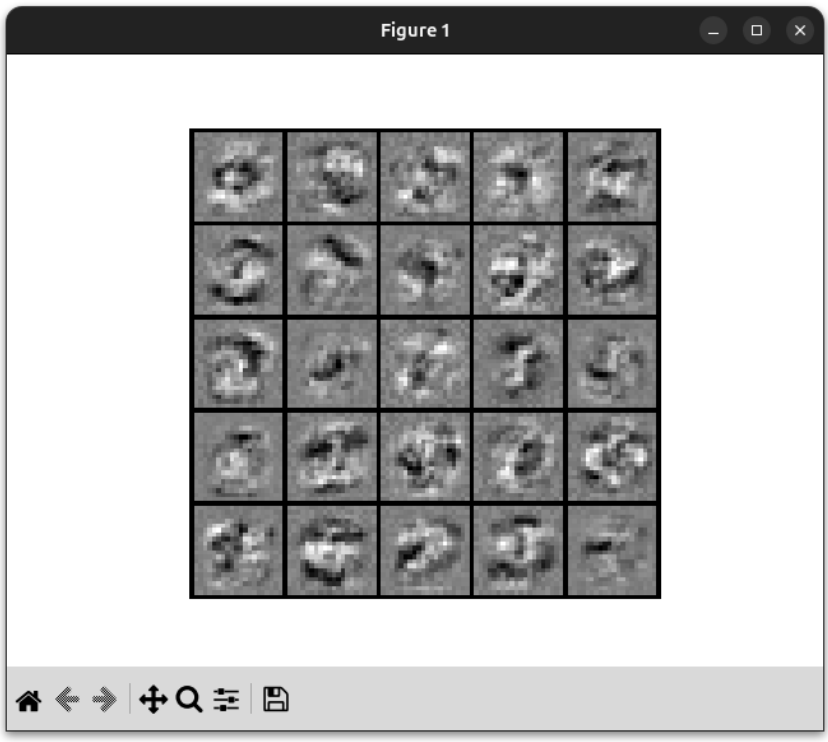
```
-------------------------

Training Neural Network...
/home/matheus/Documentos/machine_learning/ex03_startercode/venv/lib/python3.12/s
ite-packages/scipy/optimize/_optimize.py:1704: OptimizeWarning: Maximum number o
f iterations has been exceeded.
  res = _minimize_cg(f, x0, args, fprime, callback=callback, c1=c1, c2=c2,
         Current function value: 0.314928
         Iterations: 400
         Function evaluations: 964
         Gradient evaluations: 964

-------------------------

Visualizing Neural Network...
Training Set Accuracy: 99.420000
```

3.4 | 5.4 Visualising the hidden layer

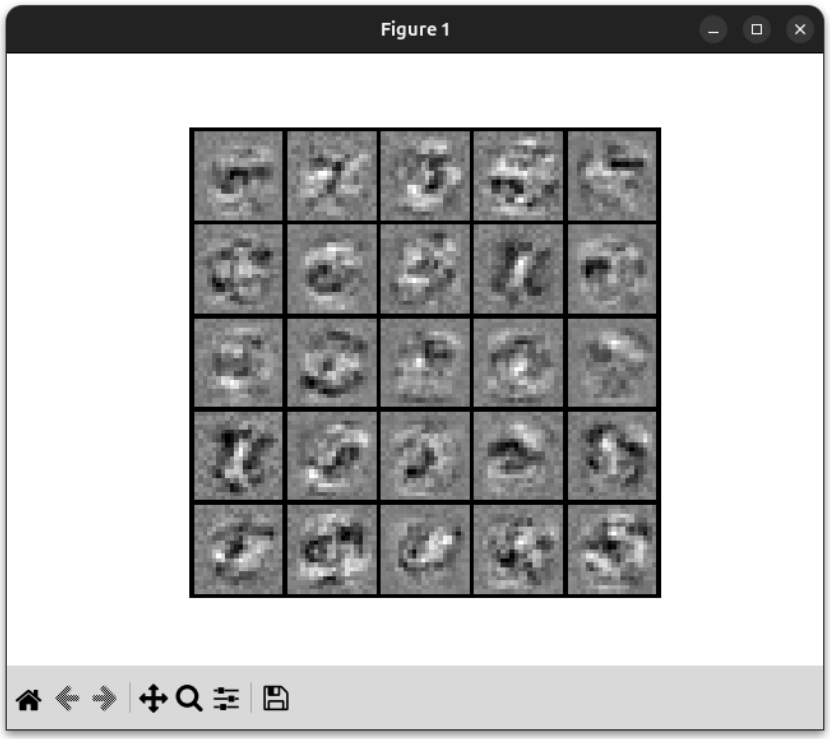


The neurons in the hidden layer learn intermediate representations of the data, such as features and patterns present in the images, and the output layer combines these representations to classify the digits.

3.5 | 5.5 Final analysis

A: $\lambda = 0$ and maxiter = 50

With $\lambda = 0$ and only 50 iterations, there is no regularization and the optimization stops relatively early. The training accuracy is about 95.88%. The network has not yet fully fitted the training data.




```
 ------------------------

Training Neural Network...
/home/matheus/Documentos/machine_learning/ex03_startercode/venv/lib/python3.12/s
ite-packages/scipy/optimize/_optimize.py:1704: OptimizeWarning: Maximum number o
f iterations has been exceeded.
  res = _minimize_cg(f, x0, args, fprime, callback=callback, c1=c1, c2=c2,
         Current function value: 0.322047
         Iterations: 50
         Function evaluations: 107
         Gradient evaluations: 107

 ------------------------

Visualizing Neural Network...
Training Set Accuracy: 95.880000
```

B: $\lambda = 0$ and maxiter = 400

With $\lambda = 0$ and 400 iterations, the network reaches 100% training accuracy. Since there is no regularization and the number of iterations is high, the network can fit the training set extremely closely. This corresponds to a high risk of overfitting.



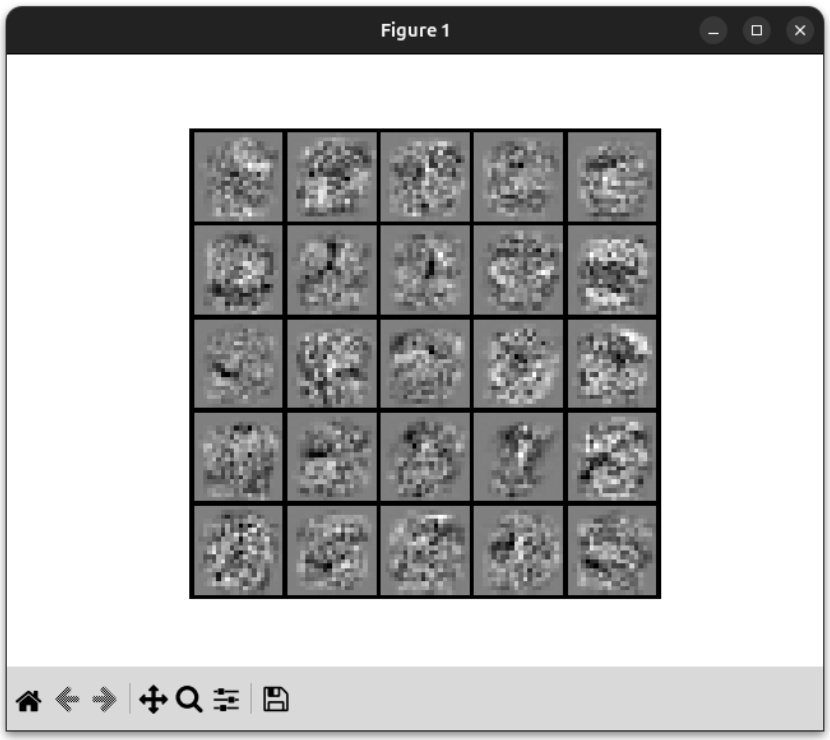

```
/home/matheus/Documentos/machine_learning/machine_learning_git/Machine_Learning/
ex03_startercode/nnCostFunction.py:86: RuntimeWarning: invalid value encountered
 in multiply
  J = (-1/m) * np.sum(y_matrix.T * np.log(a3) + (1 - y_matrix.T) * np.log(1 - a3
))
/home/matheus/Documentos/machine_learning/ex03_startercode/venv/lib/python3.12/s
ite-packages/scipy/optimize/_optimize.py:1704: OptimizeWarning: NaN result encou
ntered.
  res = _minimize_cg(f, x0, args, fprime, callback=callback, c1=c1, c2=c2,
         Current function value: nan
         Iterations: 276
         Function evaluations: 1094
         Gradient evaluations: 1094

 -------------------------

Visualizing Neural Network...
Training Set Accuracy: 100.000000
```

C: $\lambda = 0.1$ and maxiter = 400

With $\lambda = 0.1$ and 400 iterations, the network also reaches 100% training accuracy. The weak regularization still allows a very complex model, therefore the risk of overfitting remains high.




```
 -------------------------

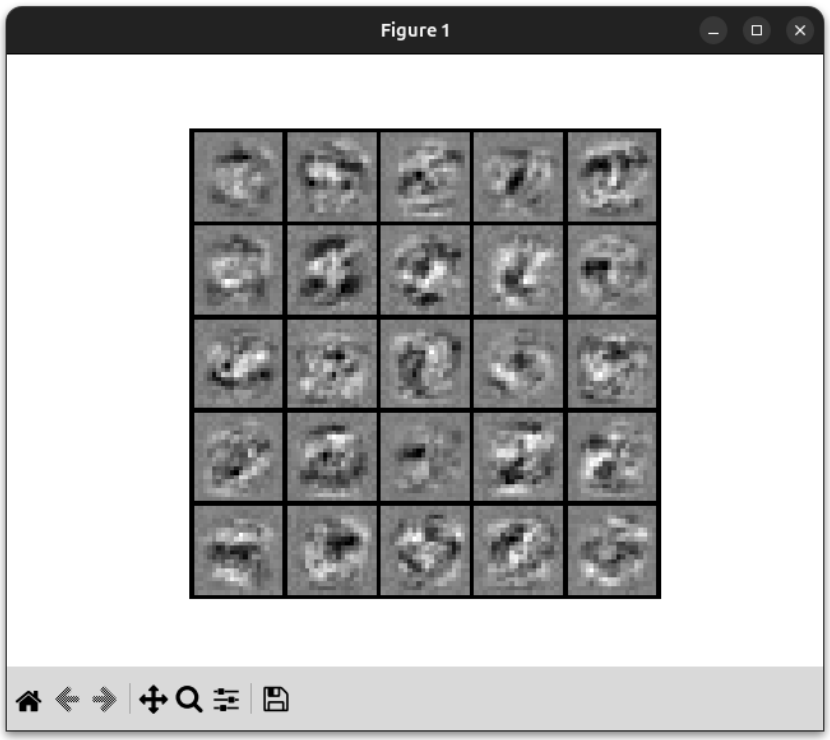
Training Neural Network...
/home/matheus/Documentos/machine_learning/ex03_startercode/venv/lib/python3.12/s
ite-packages/scipy/optimize/_optimize.py:1704: OptimizeWarning: Maximum number o
f iterations has been exceeded.
  res = _minimize_cg(f, x0, args, fprime, callback=callback, c1=c1, c2=c2,
         Current function value: 0.069574
         Iterations: 400
         Function evaluations: 1155
         Gradient evaluations: 1155

 -------------------------

Visualizing Neural Network...
Training Set Accuracy: 100.000000
```

D: $\lambda = 1$ and maxiter = 50

With $\lambda = 1$ and 50 iterations, the regularization limits the model complexity and the short training time gives an accuracy of about 96.78%. The model is less fitted to the training set than in the cases with 400 iterations.




```
 ------------------------

Training Neural Network...
/home/matheus/Documentos/machine_learning/ex03_startercode/venv/lib/python3.12/s
ite-packages/scipy/optimize/_optimize.py:1704: OptimizeWarning: Maximum number o
f iterations has been exceeded.
  res = _minimize_cg(f, x0, args, fprime, callback=callback, c1=c1, c2=c2,
         Current function value: 0.418716
         Iterations: 50
         Function evaluations: 113
         Gradient evaluations: 113

 ------------------------

Visualizing Neural Network...
Training Set Accuracy: 96.780000
```

E: $\lambda = 1$ and maxiter $= 400$

With $\lambda = 1$ and 400 iterations, the neural network reaches about 99.38% training accuracy. This shows that a moderate regularization still allows the model to fit the training data very well.




```
 -------------------------

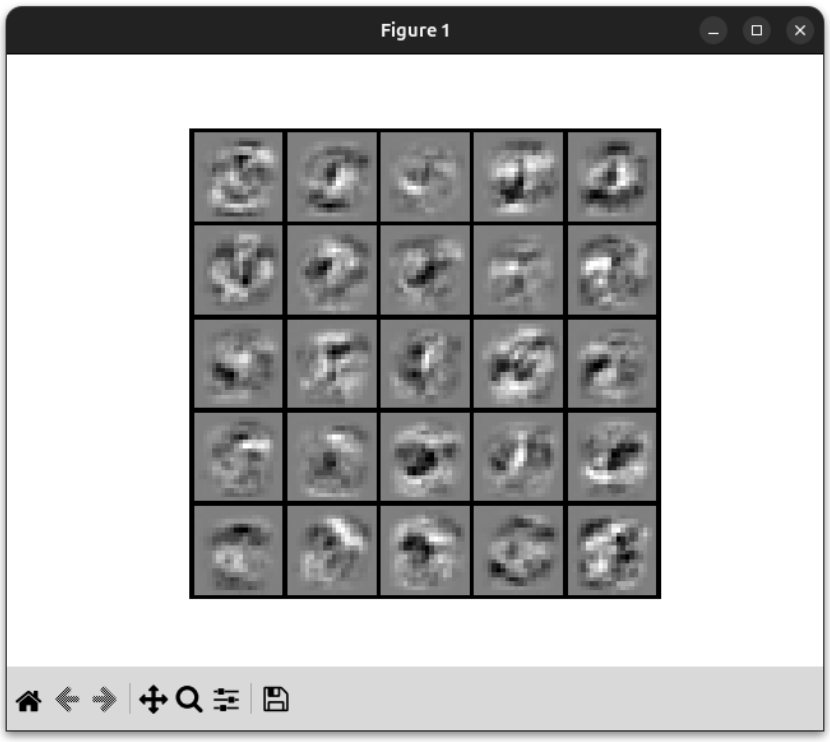
Training Neural Network...
/home/matheus/Documentos/machine_learning/ex03_startercode/venv/lib/python3.12/s
ite-packages/scipy/optimize/_optimize.py:1704: OptimizeWarning: Maximum number o
f iterations has been exceeded.
  res = _minimize_cg(f, x0, args, fprime, callback=callback, c1=c1, c2=c2,
         Current function value: 0.320238
         Iterations: 400
         Function evaluations: 910
         Gradient evaluations: 910

 -------------------------

Visualizing Neural Network...
Training Set Accuracy: 99.380000
```

F: $\lambda = 10$ and maxiter = 400

With $\lambda = 10$ and 400 iterations, the regularization becomes much stronger. The training accuracy decreases to about 94.26%, showing that the model is more constrained and less able to fit the training data.




```
 -------------------------

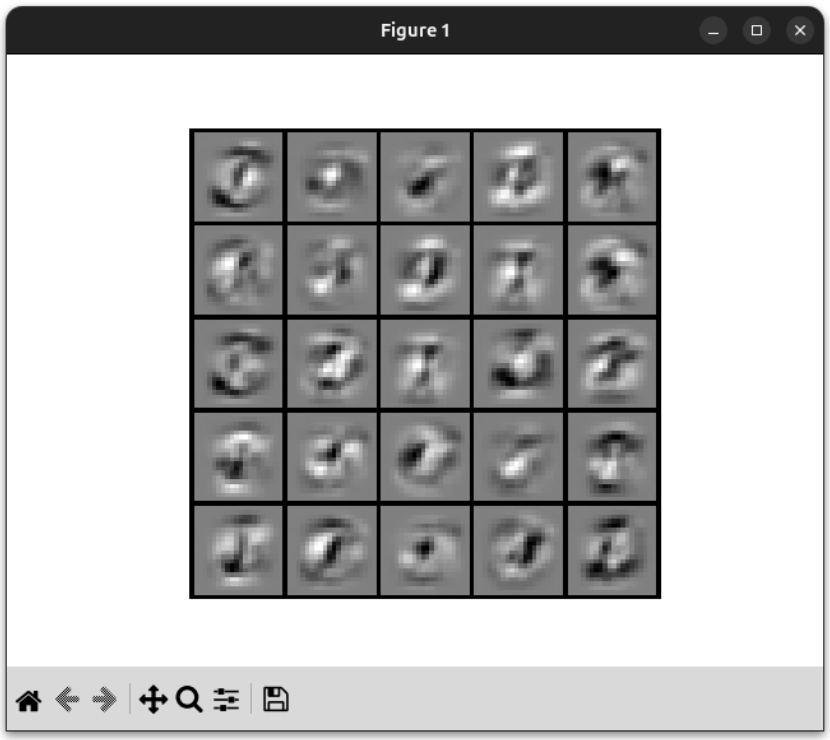
Training Neural Network...
/home/matheus/Documentos/machine_learning/ex03_startercode/venv/lib/python3.12/s
ite-packages/scipy/optimize/_optimize.py:1704: OptimizeWarning: Maximum number o
f iterations has been exceeded.
  res = _minimize_cg(f, x0, args, fprime, callback=callback, c1=c1, c2=c2,
         Current function value: 0.986321
         Iterations: 400
         Function evaluations: 800
         Gradient evaluations: 800

 -------------------------

Visualizing Neural Network...
Training Set Accuracy: 94.260000
```

G: $\lambda = 100$ and maxiter = 400

With $\lambda = 100$ and 400 iterations, the regularization is very strong and the training accuracy drops to about 84.74%. In this case, the model is too constrained and this is an example of underfitting.




```
to be about 0.576051)

 ------------------------

Training Neural Network...
Optimization terminated successfully.
         Current function value: 2.482299
         Iterations: 364
         Function evaluations: 727
         Gradient evaluations: 727

 ------------------------

Visualizing Neural Network...
Training Set Accuracy: 84.740000
```

Small values of λ give better training accuracy but increase the risk of overfitting. Large values of λ can cause underfitting.

Increasing `maxiter` generally improves the fit to the training data.

3.6 | 6 Some questions, you have to answer...

· Décrivez l'approche basée sur les réseaux de neurones. Quelle modélisation est retenue? Quels sont les paramètres à apprendre?

R: On utilise un réseau de neurones pour reconnaître les chiffres manuscrits. Le réseau contient 400 entrées, une couche cachée de 25 neurones et une couche de sortie de 10 neurones. Les paramètres à apprendre sont les poids entre les différentes couches, représentés par $\Theta^{(1)}$ et $\Theta^{(2)}$.

· Quelle est la fonction de coût? Quelles modifications sont effectuées par rapport à celle utilisée en TP2?

R: La fonction de coût est basée sur la même idée que celle de la régression logistique du TP2. La différence est que le coût est maintenant calculé pour les 10 sorties du réseau. On ajoute aussi une régularisation sur les poids, sauf pour les termes de biais.